



Java – Basics

Task

[Visit our website](#)

Introduction

Java is a high-level programming language developed in 1995 by Sun Microsystems. This general-purpose language is simple to learn, highly portable, and can run on a variety of platforms. This task provides a brief overview of both software engineering itself and the fundamental concepts of Java, such as variables, data types, and control structures.

What is software engineering?

To understand the concept of software engineering, you first need to understand what software is. You might think that a software engineer is simply concerned with computer programs that provide desired features, functions, and performance when executed. However, this is not entirely the case. A software engineer is not just responsible for computer programs, but also all associated documentation and configuration data, which ensures the correct operation of the program. Software that can be sold to a customer is known as a **software product**.

Now, what is software engineering? The IEEE (1993) defines software engineering as follows:

Software engineering: (1) The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software. (2) The study of approaches as in (1).

In simpler terms, software engineering is an engineering discipline, which is concerned with the development of a software product using well-defined scientific principles, methods, and procedures.

Software engineering vs programming

You might be tempted to use the terms *programmer* and *software engineer* interchangeably since they both develop software applications. However, this would be inaccurate. There are some distinct differences between the responsibilities and approaches to the job for these two roles.

Much like other engineering disciplines, software engineering approaches developing software as a formal process of understanding requirements, designing software, and then deploying, testing, and maintaining it. Computer programmers, on the other hand, write code to the specifications given to them. Software engineers often work as a part of the team with other engineers, business analysts, product managers, etc.

Programmers work more independently on coding, but they will also need to communicate with other team members for ideas, feedback, etc.

What makes a good software engineer?

You might assume that the most important qualities that a software engineer can possess are technical experience and knowledge, however, this is not entirely true. A good software engineer is more than the sum of the technologies with which they have worked. Below is a list of some important traits that set successful software engineers apart:

- **Passion for code:** This will come as no surprise, but a good software engineer should have a passion for programming. A software engineer should have an inherent interest in programming and enjoy working with code, as it is something that you will be working with almost every day.
- **Team player:** Very few projects are small enough to be undertaken by a single person. Therefore, software engineers often find themselves working in a team, and software engineers must have excellent teamwork skills.
- **Excellent communication skills:** Software engineers should be able to communicate successfully with team members as well as clients. They should have both great written and verbal communication skills and be able to write reports as well as give instructions.
- **Curious:** Good software engineers are curious people. They want to know why and how things happen, like how the code and conditions produce certain software behaviour. This curiosity enables these engineers to think outside the box and come up with creative solutions to problems.
- **Wide-ranging technical experience:** You can't escape the fact that a successful software engineer should have broad technical experience. Knowing a single programming language is not sufficient. A good software engineer should be well-versed in best practices like agile, task management software, version control, and working in different development environments. A good software engineer should also always be willing to learn new technologies.

This is by no means an exhaustive list of the qualities of a good software engineer. However, it gives you a sense of what it takes to become successful in this field.

The Eclipse IDE

An IDE, or integrated development environment, is an essential tool used by software engineers to develop code. It consists of a source code editor, a compiler, and a

debugger. It increases a software engineer's productivity by condensing the entire process of code creation, compilation, and testing. The Eclipse IDE has been one of the most popular IDEs for Java, with a market share of 23% in 2024. Eclipse is primarily used for developing Java applications, however, it can also be used for developing applications in other programming languages via plug-ins. These languages include JavaScript, PHP, Python, and Ruby to name a few.

Creating a simple Java program

The following Java program will display the message "Hello World!" on the console (the computer's display device):

```
public class Welcome {  
    public static void main(String[] args) {  
        System.out.println("Hello World!");  
        // Prints out Hello World!  
    }  
}
```

In this Java program, the first line defines a class named `Welcome`. Every Java program must have at least one class, and the class name should match the file name.

The second line defines the `main()` method, which is where the program starts execution. A method in Java is similar to a function in other languages: it contains statements that perform actions. A class can have multiple methods, but the `main()` method is the entry point of the program.

The `main()` method can take arguments. Here, `String[] args` is an argument that allows the program to accept input when it starts. For now, just remember that `public static void main (String[] args) { ... }` is the standard way to define the program's starting point. We will cover methods in more detail later.

Inside the `main()` method, there is a `System.out.println()` statement that prints `Hello World!` to the console. In Java, every statement ends with a semicolon (`;`), which acts as a statement terminator.

The fourth line is a comment. Single-line comments start with double slashes (`//`), while multi-line comments are enclosed within `/*` and `*/`.

All parts of the program are enclosed in curly braces (`{ }`). Each class groups its data and methods within curly braces, and each method groups its statements within its own curly braces.

Reading input from the console

Java uses `System.out` to refer to the standard output device, which is the computer's display monitor. It uses `System.in` to refer to the standard input device, which is the keyboard.

In the program above, we used the `System.out.println()` method to print a message to the console, but what if we would like to read input from the console? To do this, create a `Scanner` object to read input from `System.in`. You do this by using the following syntax:

```
Scanner input = new Scanner(System.in);
```

In the code above, `new Scanner(System.in)` creates a `Scanner` object (you will learn more about objects later), while the code `Scanner input` declares a variable of type `Scanner`. However, before creating a new `Scanner` object, we need to import the package where `Scanner` is declared. In order to do this we use the following import statement:

```
import java.util.Scanner;
```

The following is a sample code to illustrate how `Scanner` can be used:

```
/*
 * This program prompts the user to enter their name
 * and then greets them with a personalised message.
 */

import java.util.Scanner;

public class SampleApp {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.println("Please enter your name:");
        String name = input.next();
        System.out.println("Nice to meet you, " + name + "!");
    }
}
```

When the code is run, the program will print the statement "Please enter your name" to the console to ask for your name. You can enter any name and press "Enter". The name you input is stored in a variable `name`. The line `String name = input.next();` reads the next token of input (in this case, the name you enter) and stores it in the `name` variable as a string. The program then prints out a greeting along with your input name. For example:

```
Please enter your name:
Bob
Nice to meet you, Bob!
```

Variables

Variables are used to store values that you might wish to use later in the program. They are called variables because the values they store can be changed.

A variable needs to be declared before it can be used. Declaring a variable informs the compiler how much memory to allocate to the variable based on its data type. Unlike in JavaScript, to declare a variable you need to specify the data type followed by the variable name.

Examples of variable declarations can be found below:

```
int number;
double interestRate;
```

In Java, `int` and `double` are data types that specify the kind of data a variable can hold. In the code above, `number` and `interestRate` are the names of these variables.

The first statement declares a variable named `number` that can hold an integer value, as `int` is a Java data type used for storing whole numbers. The second statement declares a variable named `interestRate` that can hold a floating-point value. `double` is a Java data type used for storing decimal numbers with high precision.

Once a variable is declared, it can be assigned a value using the assignment operator (`=`). For example:

```
number = 234;
interestRate = 4.56;
```

These assignments set `number` to 234 and `interestRate` to 4.56, respectively.

Named constants

A named constant holds a permanent value that cannot be changed. To declare a constant, use the following syntax:

```
final double PI = 3.14159;
final int MAX_VALUE = 100;
```

In this example, two constants are declared: `PI`, of type `double`, with a value of `3.14159`, and `MAX_VALUE`, of type `int`, with a value of `100`.

The `final` keyword in Java is used to mark a variable as a constant, meaning it cannot be re-assigned after its initial value is set. Therefore, `PI` and `MAX_VALUE` become constants. Attempting to re-assign a value to a `final` variable results in a compilation error. For example:

```
final double PI = 3.14159;
PI = 4;
```

This would produce the following error message:

```
Exception in thread "main" java.lang.Error: Unresolved compilation problem:
    The final local variable PI cannot be assigned. It must be blank and not
    using a compound assignment
```

This error means that once a final variable has been assigned a value, it cannot be changed. The compiler prevents any reassignment to ensure the value remains constant throughout the program.

It is good practice to use capital letters when naming constants.

Numeric data types

There are six numeric data types for integers and floating-point numbers in Java.

- **Integer types:** `byte`, `short`, `int`, and `long`.
- **Floating-point number types:** `float` and `double`.

These numeric data types, as well as `boolean` and `char` types mentioned below, are primitive data types in Java. Primitive data types are the fundamental types provided by the Java language to represent simple values. They are not objects and do not have methods or properties. Instead, they store their values directly in memory. Each primitive data type has a defined range and storage size that is consistent across different platforms, ensuring predictable behavior.

For example, `int` is a 32-bit signed integer type with a value range from `-2,147,483,648` to `2,147,483,647`. Similarly, `short` is a 16-bit signed integer type with a value range from `-32,768` to `32,767`.

If you need to store whole numbers, you can use one of the `byte`, `short`, `int`, or `long` integer types. For values requiring floating-point precision, you can use `float` or `double`.

Name	Range	Storage size (bits)
byte	-128 to 127	8
short	-2^{15} to $2^{15}-1 = -32768$ to 32767	16
int	-2^{31} to $2^{31}-1 = -2147483648$ to 2147483647	32
long	-2^{63} to $2^{63}-1 = -9223372036854775808$ to 9223372036854775807	64
float	Negative range: $-3.4028235E+38$ to $-1.4E-45$ Positive range: $1.4E-45$ to $3.4028235E+38$	32
double	Negative range: $-1.7976931348623157E+308$ to $-4.9E-324$ Positive range: $4.9E-324$ to $1.7976931348623157E+308$	64

To initialise variables with these numeric data types, there are some conventions to follow as shown in example below:

```
int positiveNumber = 10001; // A positive integer value
int negativeNumber = -45; // A negative integer value
int largeNumberWithUnderscores = 9_000_000; // Integer with underscores for better readability

short smallNumber = 1234; // A short integer value

byte maxByteValue = 127; // Maximum value that a byte type can hold
byte outOfRangeByte = 128; // Error: 128 exceeds the byte type range

long longNumberWithSuffix = 14L; // Long integer with "L" suffix to denote its type
long longNumberWithoutSuffix = 14; // Equivalent to 14; automatically converted from int to long
long largeLongNumber = 50_123_456_789L; // A large long integer value with "L" suffix
long tooBigForIntConversion = 50_123_456_789; // Error: value too large for int without explicit conversion

double decimalNumber = 8.31; // Standard double-precision floating-point number
double scientificNotationPositive = 3.00e+4; // 3 * 10^4, which equals 30,000
double scientificNotationNegative = 1e-6; // 1 * 10^-6, which equals 0.000001
```

```
float floatNumberWithSuffix = 8.31F; // Float with "F" suffix to denote its
type
float scientificNotationFloat = 3.00e+8F; // 3 * 10^8, which equals
300,000,000
float piValueWithUnderscores = 3.1415_9265F; // Float value with underscores
for readability
```

Casting

You can convert one data type to another using casting. There are two types of casting in Java: implicit and explicit.

Implicit casting occurs automatically when you assign a value to a variable whose type supports a larger range of values. This is known as widening a type.

For example, you can implicitly cast an integer (`int`) to a floating-point number (`float`) because a `float` can represent a wider range of values and more precise values than an `int`.

Here's an example of implicit casting:

```
int number1 = 5;
float number2 = number1;
```

Casting a type with a large range to a type with a smaller range is known as narrowing a type. Use **explicit casting** to do this. To explicitly cast a type, specify the target type in parentheses, followed by either the variable's name or value you wish to cast.

An example of explicit casting is as follows:

```
int number1 = (int) 23.34;
```

```
long longNum = 656666L;
int intNum = (int) longNum;
```

The character and string data types

In Java, you can represent a single character using the character data type `char`. All character values must be enclosed between single quotation marks. For example:

```
char letter = 'N';
char number = '5';
```

You can also represent a string of characters using the `String` data type. Like many other programming languages, all strings must be enclosed between double quotation marks. For example:

```
String newString = "Java is fun!";
```

The Boolean data type

In Java, as in JavaScript, the `boolean` data type can only hold one of two possible values: `true` or `false`. Boolean values often result from comparison operations and are used to represent logical conditions.

For example, you can initialise Boolean variables like this:

```
boolean isPrimeNumber = false; // Indicates that the number is not prime  
boolean isVehicle = true;      // Indicates that the object is a vehicle
```

Boolean values can also be derived from comparison operations. For instance, the following statement checks if a number is less than 100 and prints the result:

```
int number = 10;  
System.out.println(number < 100); // Prints "true" because 10 is less than 100
```

In this case, `number < 100` evaluates to `true`, which is then printed to the console.

If-else statements

As you know, selection statements play a pivotal role in programming. Selection statements allow the program to decide which statements to execute based on a condition. An *if statement* will execute a block of code only if the condition is true. The syntax for an *if statement* in Java is as follows:

```
if (boolean expression) {  
    statement(s);  
}
```

What happens if the condition is false? If you had an *if* statement, nothing would happen. If you want to take alternative actions when the condition is false, add an *else* statement to create an *if-else* statement. If the condition turns out to be false, the statements indicated by the *else* statement will be executed. The syntax for an *if-else* statement is as follows:

```

if (boolean expression) {
    statement(s); // Executed if Boolean expression evaluates to true
}
else {
    other statement(s); // Executed if Boolean expression evaluates to false
}

```

An if or if-else statement can also be placed within another if or if-else statement to form a nested statement.

For example, the following code tests what symbol a student should be awarded based on their class mark. If the student achieves a mark of 80 or higher, they receive an A, if they achieve a mark of 70 or greater, they receive a B, and so on.

```

int mark = 75; // Example mark
char grade;

if (mark >= 80) {
    grade = 'A';
} else if (mark >= 70) {
    grade = 'B';
} else if (mark >= 60) {
    grade = 'C';
} else if (mark >= 50) {
    grade = 'D';
} else {
    grade = 'F';
}

System.out.println("The grade is: " + grade); // Outputs the grade based on the mark

```

The while loop

Loops are used to control how many times a statement or sequence of statements is executed. The while loop is a simple loop that executes statements repeatedly while a condition is true.

The syntax for the while loop is:

```

while (continuation condition) {
    statement(s);
}

```

To avoid an infinite loop (a loop that never terminates), you must make sure that the continuation condition will eventually become false and the loop stops running.

For example, the following code calculates the sum from 1 to 10.

```
// This code calculates the sum of numbers from 1 to 10 using a while loop.

int sum = 0;
int count = 1;
while (count <= 10) {
    sum += count;
    count++;
}
System.out.println(sum); // Prints out 55
```

In this example, the continuation condition is the variable `count` being smaller or equal to 10. The variable `count` is initialised as 1. There is a statement `count++` to increment this variable by 1 in the `while` loop, so that the continuation condition will become false after the `while` loop has iterated 10 times. This is because, by that time, the variable `count` is 11, so the continuation condition is evaluated as false. Thus, the `while` loop exits.

The for loop

The `for` loop provides a concise, simple way to iterate over a range of values. The syntax for the `for` loop is:

```
for (initialisation; termination; update) {
    statement(s);
}
```

The `initialisation` expression is executed once when the loop begins. It is normally used to initialise a control variable. Next, the `termination` expression tests whether the control variable has reached its termination value. If not, the statements in the body of the loop are executed. The `update` expression then increments or decrements the control variable and the termination value is tested once again. The process is repeated until the variable has finally reached its termination value.

For example, the following code calculates the sum from 1 to 10 using a `for` loop.

```
// This code calculates the sum of numbers from 1 to 10 using a for loop

int sum = 0;
for (int count=1; count <= 10; count++) {
    sum += count;
}
```

```
System.out.println(sum); // Prints out 55
```

The count is initialised as 1 in the initialisation part of the for loop. The termination condition is `count <= 10`. The update statement is that count is incremented by 1 in each iteration.

Similarly, you can calculate the sum from 10 to 1 by initialising count as 10 and having a decrementing update statement.

```
// This code calculates the sum of numbers from 10 to 1 using a for loop,  
with count starting at 10.
```

```
int sum = 0;  
for (int count=10; count >= 1; count--) {  
    sum += count;  
}  
System.out.println(sum); // Prints out 55
```

The following code shows you a nested for loop example that prints out a triangle:

```
/*  
 * This program prints out a triangle pattern using nested for loops.  
 * Each row of the triangle has an increasing number of asterisks (*).  
 */  
  
int rowCount = 6;  
for (int i = 1; i <= rowCount; i++) {  
    for (int j = 1; j <= i; j++) {  
        System.out.print("* ");  
    }  
    System.out.println();  
}
```

This code will print the following pattern:

```
*  
* *  
* * *  
* * * *  
* * * * *  
* * * * * *
```

Explanation:

When the outer `for` loop (the loop with variable `i`) runs, it iterates over `i` with values from 1 to `rowCount` (which is 6). So, there are 6 iterations of the outer loop in total.

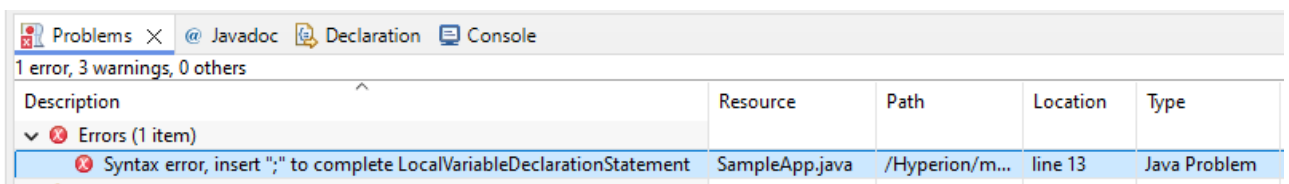
- During the first iteration of the outer loop, `i` is 1. Then, in this first iteration, the inner `for` loop (loop with variable `j`) starts to run. This inner loop iterates over `j` with values from 1 to `i` (which is 1). Since the termination condition is `j <= i`, the inner loop iterates once, printing a single `*`, which forms the first row of the triangle.
- During the second iteration of the outer loop, `i` is incremented by 1 (because of the `i++` increment statement), making `i` equal to 2. The inner `for` loop runs again, this time iterating over `j` with values from 1 to `i` (which is 2). The termination condition `j <= i` means the inner loop iterates twice, printing two `*` characters, forming the second row of the triangle.

This process continues, with the number of `*` characters in each row matching the current value of `i` in the outer loop, resulting in a triangle pattern.

Dealing with errors

Everyone makes mistakes. Maybe you've made some already. It's important to be able to **debug** your code. As you learned with JavaScript, as you code, you will encounter the following types of errors:

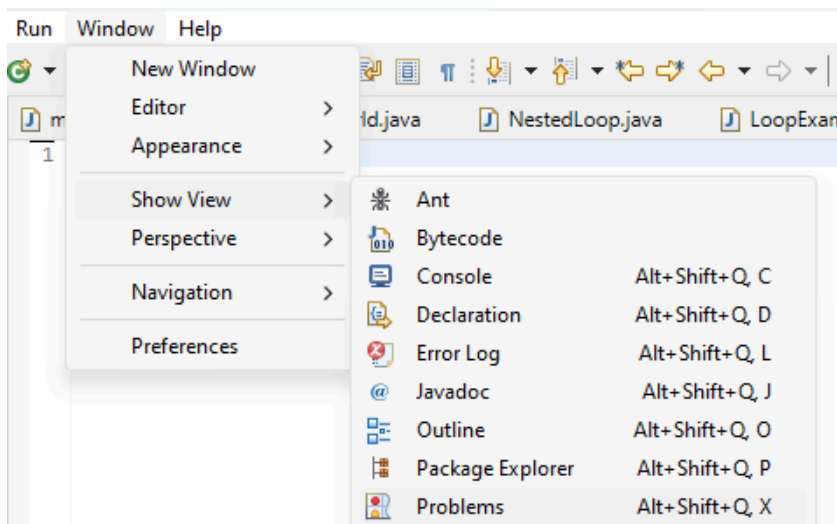
- **Compilation errors:** Compilation errors occur when the code does not conform to the syntax rules of the programming language. When you attempt to compile your code, errors may appear in the “Problems” panel of Eclipse, or a similar panel in other IDEs. These errors can result from issues such as missing semicolons or syntax mistakes. Below is an example of an error you might encounter during compilation:

A screenshot of the Eclipse IDE's 'Problems' panel. The panel has tabs for 'Problems', 'Javadoc', 'Declaration', and 'Console'. The 'Problems' tab is active, showing '1 error, 3 warnings, 0 others'. Below this, there is a table with columns: 'Description', 'Resource', 'Path', 'Location', and 'Type'. The table contains one entry: 'Syntax error, insert ";" to complete LocalVariableDeclarationStatement' for 'SampleApp.java' at 'line 13', categorized as a 'Java Problem'.

Problems				
1 error, 3 warnings, 0 others				
Description	Resource	Path	Location	Type
▼ Errors (1 item)				
✖ Syntax error, insert ";" to complete LocalVariableDeclarationStatement	SampleApp.java	/Hyperion/m...	line 13	Java Problem

This means that on line 13 of the file **SampleApp.java**, you forgot to insert a semicolon. To fix the error, double-click on the error in the “Problems” panel, and then the Eclipse editor will open the file at the specific line for you to edit.

You can open the “Problems” panel (if it is not opened yet) from *Window > Show View > Problems*:



- **Runtime errors:** Runtime errors occur when your program compiles successfully but encounters problems while running, causing it to stop abruptly. These errors are displayed in the console when the program is executed. For example, if your program contains a line that causes an error during execution, such as accessing an invalid array index or dividing by zero, the console will display the error message, helping you identify what went wrong. Consider the following code as an example:

```
int a = Integer.parseInt("18.2");
```

Your code would compile properly, but when running you'd get the error:

```
Exception in thread "main" java.lang.NumberFormatException: For
input string: "18.2"
    at
java.lang.NumberFormatException.forInputString(NumberFormatExcepti
on.java:65)
    at java.lang.Integer.parseInt(Integer.java:492)
    at java.lang.Integer.parseInt(Integer.java:527)
    at example.main(example.java:107)
```

The error `java.lang.NumberFormatException` indicates a problem with the format of the input string. In this case, the string `"18.2"` cannot be parsed as an integer because integers cannot include decimal points. Understanding the type of exception helps in diagnosing and fixing runtime errors. It's crucial to analyse the error messages and deduce the cause of the problem to resolve runtime issues effectively.

- **Logical errors:** Logical errors occur when your program compiles and runs, but the output does not match your expectations. This type of error results from incorrect logic or flawed algorithms in your code, often referred to as a bug.

Logical errors can be particularly challenging to identify and fix, especially when dealing with loops and control statements. Even experienced programmers may spend significant time designing and debugging algorithms to ensure they work as intended.

For instance, consider the following code, which is intended to calculate the sum of numbers from 1 to 10:

```
int count = 1;
int sum = 0;
for (count = 1; count <= 10; count++); // A misplaced semicolon causes
a logical error
{
    sum += count; // Incorrectly accumulates values
}
System.out.println(sum); // Incorrectly prints 11 instead of 55
```

In this example, the semicolon at the end of the `for` loop statement creates an empty loop body, causing the sum to be updated incorrectly. As a result, the output is **11** instead of the expected **55**. Identifying and fixing such logical errors requires careful analysis of the code to ensure that the logic correctly implements the intended algorithm.

Practical task 1

This task will walk you through how to install Eclipse.

- To use Eclipse for Java, you will first need to install the Java Development Kit (JDK). If you do not have it already installed you can find a tutorial on how to install it on [this website](#).
- Once the JDK is installed, you can download Eclipse from the [Eclipse downloads page](#) by following these instructions:
 - Under “Install your favorite desktop IDE packages” select “Download Packages”.
 - Find “Eclipse IDE for Java Developers” in the download package page.
 - Click the link of your operating system (Windows, macOS, or Linux) version.
 - You then click on the “Download” button to begin your download.

- Once the file has downloaded, unzip it into a directory of your choice and click on the **eclipse.exe** file (if using Windows) to start the IDE.
- When you have successfully downloaded and installed Eclipse, you will be required to select a workspace directory. Once you have chosen a directory, select "OK".
- You are then greeted by the welcome screen. Take a screenshot of this screen and send it to us to show that you have successfully installed Eclipse.

If you are having any difficulties installing or downloading Eclipse or the JDK, please feel free to contact an expert code reviewer.

Be sure to place files for submission inside your task folder and click "Request review" on your dashboard.

Practical task 2

Create a dice rolling game by following these steps:

- Create a new file called **HighRoller.java**.
- This game will involve you competing against the computer. Both you and the computer will roll a six-sided dice to determine who gets the higher number. The player who wins **N** rounds first (where **N** is a positive integer input by you) will be the overall winner.
- For example if **N** = 3, and the dice rolls are as follows:
 - **1st round:** You roll 4, Computer rolls 3.
 - **2nd round:** You roll 3, Computer rolls 1.
 - **3rd round:** You roll 2, Computer rolls 2.
 - **4th round:** You roll 6, Computer rolls 4.

After the 4th round you win the game because you reach the first 3 wins earlier.

- When either you or the computer wins the game, display the winner along with the number of wins, draws, and losses. Then, the game should exit.
- The program should start by asking you to input a positive integer of **N** in the console. (**Hint:** Ensure that **N** is a valid positive integer.)

- The program should randomly generate a number between 1 and 6 to simulate rolling a six-sided die. (**Hint:** Utilise `Random.nextInt(6)` to simulate rolling a 6-sided die.)
 - Please read [this webpage](#) to understand how to use the `Random` class's `nextInt(bound)` method to generate a random integer.
 - For example, to generate a random integer between 1 and a specific upper bound, such as 1000, you can use the following code:

```
Random random = new Random();
```

```
int randomNumber = random.nextInt(1000) + 1;
```

- The program should then prompt the player to “roll their dice”.
- Once both you and the computer have rolled their dice, compare the results. Display the outcomes of each round, whether you won, lost, or tied based on the higher roll.

Be sure to place files for submission inside your task folder and click "Request review" on your dashboard.



Share your thoughts

HyperionDev strives to provide internationally excellent course content that helps you achieve your learning outcomes.

Do you think we've done a good job or do you think the content of this task, or this course as a whole, can be improved?

Share your thoughts anonymously using this [form](#).
